

Aligned Fragments and Constrained Crossover

Undecidability, exact generation depth,
and rational growth of context-free recombination closures

Research draft prepared with ChatGPT
for Vladimir Reshetnikov

29 September 2026

Abstract

For equal-length words, constrained one-point crossover exchanges suffixes at the same position. We prove that deciding whether a context-free language satisfies $L \otimes_c L = L$ is co-recursively-enumerable-complete, already over a fixed three-letter alphabet. This answers the decision question explicitly left open in Section 4 of Hughes, arXiv:2608.27755v1. A delimiter amplification also proves undecidability of eventual finite stabilization, with an exact rank- $r + 1$ witness built from r omitted blocks.

The structural invariant is the least number $\rho_L(w)$ of consecutive fragments of w that occur at the same positions in seed words of the same length. For parallel self-crossover we prove $\rho_{L \otimes_c L}(w) = \lceil \rho_L(w)/2 \rceil$ and hence exact generation depth $\lceil \log_2 \rho_L(w) \rceil$. Rank spectra have no gaps, and every length- n slice stabilizes by $\lceil \log_2 n \rceil$, sharply.

For regular inputs, one-step closure has a polynomial-time DFA test and is PSPACE-complete for NFAs; finite stabilization reduces to the classical limitedness problem for distance automata. For every context-free seed, the full closure has an effectively rational commutative generating function. Nevertheless, an explicit binary linear context-free seed has a closure that is not context-free. Its finite generations have polynomial growth and minimal recurrence order $q2^k + 1$, whereas the limit has exponential growth. Complete proofs, exact finite audits, and a research agenda are included.

Status. Unrefereed research draft. The proofs are mathematical arguments, not Lean or Rocq certificates. A bounded literature search supports the connection to the stated open question, not a guarantee of first discovery for every structural lemma or consequence.

[write] A report of the ProveIt research-report collection (category automata-and-formal-languages, report constrained-crossover-closure), written from batch-44 manuscript 02; text marked [write] was added then. The sibling report on insertion-degree spectra is related in Section 1.4; provenance is in Appendix D.

Contents

1 Problem, provenance, and principal results	2
1.1 A specific question behind the repository	2
1.2 What the theorems say	2
1.3 Attribution and limits of the novelty claim	3
1.4 [write] Relation to the sibling report on insertion-degree spectra	3
1.5 [write] Notation beside the neighbouring texts	4
2 Conventions and the coordinate hull	4
3 Aligned fragments and exact generation depth	6
3.1 The fragment invariant	6
3.2 There are no missing finite ranks	7
3.3 Sharp stabilization bounds	7
3.4 The frozen-source convention is also exactly solvable	8
4 One-step equality is undecidable for context-free inputs	8
4.1 A guard construction with universal first generation	9
5 Finite stabilization and an exact block amplification	10
6 Polynomial-time rank computation for a fixed target	11
7 Regular inputs: finite-state tests and limitedness	12
7.1 One-step closure for deterministic automata	12
7.2 Encoding all fragment ranks by a distance automaton	13
8 Rational counting for the full closure of every CFL	14
8.1 Effective semilinear position supports	14
8.2 A rational commutative generating function	15
9 A linear context-free seed with a non-context-free closure	16
9.1 Definition, grammar, and position geometry	16
9.2 Every rank, every generation, and exact counts	18
9.3 Sharp recurrence orders of the finite generations	18
10 Verification, reproducibility, and proof boundaries	19
10.1 Executed exact audits	19
10.2 Files and rebuilding	20
10.3 What was not verified or claimed	21
11 Further research questions and a formalization plan	21
11.1 A realistic Lean development	23
12 Conclusion	23
A The classical universality input	23
B Why the positional counts are eventually affine	24
C A compact proof-dependency map	25
D [write] Provenance of this report	25

1 Problem, provenance, and principal results

1.1 A specific question behind the repository

The ProveIt report *Gaps in Bounded-Shuffle Hierarchies*, including its later binary-profile extension, studies insertion-degree spectra and cites Hughes’s recent work on language operations [2, 1]. The report distinguishes its spectrum results from the surrounding decision problems. We follow that connection, but study a different operation: *equal-length, aligned, one-point crossover*.

Hughes defines constrained crossover on page 2 and, on page 3, explicitly leaves the decidability of $L \otimes_c L = L$ open for context-free languages. The obstruction to his general theorem is precise: constrained crossover does not contain concatenation. Our reduction does not try to restore concatenation. Instead, it makes every omitted word obtainable by a single aligned repair using a fresh guard letter. The decision problem is thereby settled directly.

[write] *Where the question is posed.* Checked when this report was written (29 September 2026) against the 13-page PDF of arXiv:2608.27755v1, whose printed page numbers are its PDF page numbers. Hughes defines unconstrained crossover $\otimes_u$ and constrained crossover $\otimes_c$ (the same symbol $\otimes_c$ as here) in Section 2 (“Definitions and Notations”, p. 2). The question is posed in Section 4 (“Foundational Results”), under “Single step equality”. His Theorem 1 makes single-step equality $L \odot L = L$ at least as hard as universality for every operator $\odot$ with $L \cdot L \subseteq L \odot L$ (on a class of languages for which containment of every letter and of ε is decidable but universality is not), and its Corollary applies this to simple insertion, shuffle and unconstrained crossover. The note after the Corollary (p. 3) observes that constrained crossover fails this subsumption criterion, so “the complexity of the question, ‘does $L \otimes_c L = L$ ’, is open.” The list of open questions that follows repeats it as its second item, for a context-free L (top of p. 4). Theorem 4.2 answers that question: the problem is co-r.e.-complete (Π_1^0 -complete), hence undecidable, already over a fixed three-letter alphabet. The first item of Hughes’s list, whether the shuffle of two context-free languages is context-free, is not addressed here. Hughes’s Section 14 (p. 12) also lists “Study constrained crossover” among its open directions without posing a specific question; Section 11 records how the present results bear on his other Section 14 questions.

The inspected ProveIt revision is

9b24a3a8d545af9624f6ac455f5b548be62818b6.

The relevant source is

SetTheory/Cardinals/docs/reports/
automata-and-formal-languages/insertion-degree-spectra/article.tex.

At this revision the source has both the original spectrum paper and its binary-profile Part II. We do not reprove their arbitrary-spectrum realization as a claimed contribution, and we do not resolve their both-singleton insertion conjecture. No theorem in the present paper assumes the correctness of an unverified ProveIt mathematical claim. [write] That source is the sibling report *insertion-degree-spectra* of the same collection category; its directory did not change between the pin and the placement of this report (commit 203016015). Section 1.4 states how the two reports relate.

1.2 What the theorems say

The results have three interacting parts. The first is a precise resource calculus: each generation halves an intrinsic fragment rank. The second is an algorithmic boundary: global stabilization is undecidable for

grammars but decidable for finite automata. The third is an enumeration theorem: the commutative counting series of a context-free seed’s full recombination closure is rational even when the resulting language is not context-free.

Question	Result proved here	Section
Exact generation of a word	$L^{[k]} = \{w : \rho_L(w) \leq 2^k\}$; a no-gap rank spectrum	3
One-step equality for CFGs	Co-r.e.-complete over a fixed ternary alphabet	4
Eventual equality for CFGs	Undecidable over four letters; strictness witnesses at every stage	5
A fixed target and grammar	Exact rank and generation decidable in polynomial time	6
Regular-input decision problems	DFA polynomial one-step test; NFA PSPACE-completeness; decidable finite stabilization	7
Counting the full closure of a CFL	Effective rationality, including commuting letter weights	8
Closure need not be context-free	Explicit linear seed; exact counts and sharp recurrence orders	9

1.3 Attribution and limits of the novelty claim

The conjectural-status claim is deliberately narrow: the one-step decision question is explicitly open in the inspected Hughes version. The proofs below answer that question under exactly its equal-length, equal-cut definition. The coordinate-product closure is an elementary recombination fact; it is reproved for completeness and is not advertised as a first discovery. Nor are context-free universality, effective Parikh semilinearity, Presburger counting, or distance-automaton limitedness new here.

The proposed contribution consists of the guarded reduction, its exact amplification, the quantitative fragment calculus and algorithms, and the rational-growth consequences in this setting. Equivalent formulations in recombination, splicing, or genetic-algorithm literature have not been exhaustively excluded. The article gives proofs so that its mathematical content can be assessed independently of priority. The source search and verification boundary are recorded again in Section 10.

1.4 [write] Relation to the sibling report on insertion-degree spectra

The sibling report *Gaps in Bounded-Shuffle Hierarchies* (ProveIt path SetTheory/Cardinals/docs/reports/automata-and-formal-languages/insertion-degree-spectra/) and this report start from the same preprint of Hughes, but from different parts of it.

- The sibling’s Part I answers questions of Hughes’s Sections 10–11. It refutes the insertion-degree no-gap conjecture (every finite set containing 1 is a spectrum), answers the finite representation-erasure question with a binary example, and proves an iteration-depth no-gap theorem (its Theorem 8.2). Its Part II realizes every finite spectrum containing 1 over a binary alphabet with one inserted unary word. Its scope sentence, under its abstract, says that it addresses “the indicated no-gap questions, not the surrounding undecidability results”. It never names the crossover question: the word “crossover” occurs there only in the title of Hughes’s preprint.
- This report answers the single-step question for constrained crossover posed in Hughes’s Section 4 (Section 1 above), one of those surrounding decision problems, for an operation the sibling does not study.
- No theorem of either report is used by the other, and none is proved in both. The no-gap theorem for aligned rank (Theorem 3.4) is an analogue, for a different operation, of the sibling’s iteration-depth no-gap theorem; neither implies the other. The sibling iterates $X_{m+1} = A \sqcup X_m$ with the inserted language A

held fixed. Its closest crossover counterpart is therefore the frozen-source iteration of Theorem 3.8, not the parallel iteration of Definition 2.1.

- Nothing here bears on the questions the sibling leaves open, in particular Hughes’s singleton insertion-degree no-gap conjecture (his Conjecture 2, the sibling’s Question 21.1), as the paragraph after Theorem 3.4 already says.

The sibling report carries a dated remark, under its scope sentence, that points to this report.

1.5 [write] Notation beside the neighbouring texts

No symbol of the manuscript was renamed. The table lists the readings that Hughes’s preprint or the sibling report could suggest.

Here	Meaning here	Not to be confused with
$A \otimes_c B$	constrained one-point crossover (Definition 2.1); the same operation and symbol as Hughes’s $\otimes_c$	Hughes’s unconstrained crossover $\otimes_u$, which contains concatenation and is covered by his Theorem 1
$T(L), L^{[k]}$	parallel self-crossover: both parents from generation k	Y_k , with one parent frozen in L (Theorem 3.8). Hughes’s iterated insertion starts from B and applies A on the left each time, and his iterated crossovers are defined “as with other operations”; for $A = B = L$ that reading gives Y_k , not $L^{[k]}$
$\rho_L(w)$	aligned-fragment rank (Definition 3.1)	the sibling’s insertion degree $d_{A,B}(w)$, a number of inserted pieces; and the unspecified “rank condition” of Hughes’s Section 14
$d_L(w)$	generation depth $\lceil \log_2 \rho_L(w) \rceil$ (5)	the sibling’s $d_{A,B}(w)$ (a degree, not a depth) and its iteration depth (a number of applications of $A \sqcup \cdot$)
“no gaps”	the ranks on a slice form $\{1, \dots, R_L(n)\}$ (Theorem 3.4)	the sibling’s spectra $\mathcal{D}(A, B)$, which <i>can</i> have gaps, and its iteration depths, which cannot
stabilization	some pair of consecutive generations is equal (Corollary 3.5)	Hughes’s “adjacent equality”; by Lemma 2.2 (and likewise for Y_k) adjacent equality here is already his “fixed point”
co-r.e.	negative instances effectively enumerable; co-r.e.-complete means Π_1^0 -complete	r.e. (the positive instances)

2 Conventions and the coordinate hull

All alphabets are finite. Put $\mathbb{N} = \{0, 1, 2, \dots\}$ and write ε for the empty word. For a word w of length n and $0 \leq i \leq j \leq n$, $w[i : j]$ denotes positions $i + 1, \dots, j$. Thus the indexing of boundaries is zero-based, but the indexing of letters is one-based. Write $L_n = L \cap \Sigma^n$.

Definition 2.1 (Constrained crossover). For languages $A, B \subseteq \Sigma^*$, define

$$A \otimes_c B = \{wz, yx : wx \in A, yz \in B, |w| = |y|, |x| = |z|\}.$$

For self-crossover put $T(L) = L \otimes_c L$. Its *parallel iterates* are

$$L^{[0]} = L, \quad L^{[k+1]} = T(L^{[k]}).$$

Both parents in generation $k + 1$ may be chosen from generation k .

The two length equalities imply that the complete parents have the same length and that their cut positions agree. Cuts at either endpoint are allowed. On a fixed slice,

$$T(L)_n = \bigcup_{i=0}^n \text{Pref}_i(L_n) \text{ Suff}_{n-i}(L_n). \quad (1)$$

The prefix and suffix in this product may come from different parents, but both parents must lie in L_n .

Lemma 2.2. *The map T is monotone and inflationary. Consequently the iterates increase, and equality at one pair of consecutive generations implies equality at all subsequent generations.*

Proof. Every pair of parents available in a smaller language is available in a larger one. Also, crossing a word with itself returns it. Thus $L \subseteq T(L)$. If $L^{[k]} = L^{[k+1]}$, applying the same map T to both sides proves the next equality, and induction finishes the argument. $\square$

Remark 2.3 (Do not change the iteration rule). The iteration $X_{k+1} = L \otimes_c X_k$, with one parent frozen in the original language, is not the iteration in Definition 2.1. In particular its resource accounting need not double each generation. The self-operation above is inflationary even though an iteration involving two different input languages can lose words of lengths absent from one input. These distinctions remove any ambiguity suggested by general remarks about nonmonotone constrained crossover.

[write] The general remark meant is in Hughes's Section 2 (p. 2): the series produced by iterated constrained crossover, unlike that of unconstrained crossover, can drop earlier elements from later sets. With two different inputs, as in $X_{m+1} = A \otimes_c X_m$ with $X_0 = B \neq A$, it does happen: the words of B whose length no word of A has are lost at the first step. For self-crossover it cannot happen under either clock: the parallel iterates increase by Lemma 2.2, and the frozen-source iterates of Theorem 3.8 increase by its formula.

Definition 2.4 (Coordinate hull). For $n \geq 1$ and $1 \leq i \leq n$, let

$$P_{n,i}(L) = \{u_i : u \in L_n\}.$$

Define $\text{Rec}(L)_n = \emptyset$ if $L_n = \emptyset$, and otherwise define

$$\text{Rec}(L)_n = P_{n,1}(L)P_{n,2}(L) \cdots P_{n,n}(L).$$

At length zero put $\text{Rec}(L)_0 = L_0$. Finally $\text{Rec}(L) = \bigcup_{n \geq 0} \text{Rec}(L)_n$.

The explicit convention at length zero is necessary. The empty product of coordinate sets must not introduce ε when the seed does not contain it. Every crossover preserves membership in the coordinate hull, since it introduces no new letter at any position. The converse reachability statement will follow from a stronger exact-rank theorem.

3 Aligned fragments and exact generation depth

3.1 The fragment invariant

Definition 3.1 (Aligned fragment and rank). Fix $w \in \Sigma^n$, $n \geq 1$. A nonempty interval $[i, j]$ is L -compatible for w if there is $u \in L_n$ with $u[i : j] = w[i : j]$. The rank $\rho_L(w)$ is the least positive integer r for which there is a partition

$$0 = i_0 < i_1 < \dots < i_r = n \quad (2)$$

whose r intervals are L -compatible. If there is no such partition, put $\rho_L(w) = \infty$. Set $\rho_L(\varepsilon) = 1$ when $\varepsilon \in L$, and ∞ otherwise.

The seed witnessing one fragment need not witness another. However, every witness is a complete seed word of length n , and every fragment is at its *original position*. Ordinary factor occurrence without alignment would give a different and incorrect invariant.

We have $\rho_L(w) = 1$ exactly when $w \in L$. For nonempty w , finite rank is at most $|w|$, because one may split any compatible partition into single-letter intervals. The existence of such a singleton partition is exactly membership in the coordinate hull.

Theorem 3.2 (Exact rank contraction). *For every language L and every word w ,*

$$\rho_{T(L)}(w) = \left\lceil \frac{\rho_L(w)}{2} \right\rceil, \quad (3)$$

where the right side is ∞ when $\rho_L(w) = \infty$.

Proof. First suppose w is nonempty. A fragment matching a word of $T(L)$ crosses at most one cut of its two seed parents. Splitting the fragment at that cut, when the cut lies strictly inside it, gives at most two nonempty L -compatible fragments. A partition of w into s $T(L)$ -compatible fragments therefore yields an L -compatible partition of size at most $2s$. In particular finite $\rho_{T(L)}(w)$ implies finite $\rho_L(w)$ and

$$\rho_{T(L)}(w) \geq \lceil \rho_L(w)/2 \rceil.$$

Conversely, take a shortest partition (2) of size r . Group consecutive fragments in pairs. If two adjacent fragments are witnessed by $u, v \in L_n$, cross u with v at their common boundary. The resulting word agrees with w throughout their union. A leftover final fragment already matches a seed, and seeds belong to $T(L)$. Thus the grouped partition has $\lceil r/2 \rceil$ $T(L)$ -compatible pieces. This proves the reverse inequality. The same argument rules out a finite rank on either side when the other side is infinite. For $w = \varepsilon$, crossover neither creates nor removes the empty word, so the convention gives the stated identity. $\square$

Theorem 3.3 (Generation formula and full closure). *For every $k \geq 0$,*

$$L^{[k]} = \{w : \rho_L(w) \leq 2^k\}, \quad \bigcup_{k \geq 0} L^{[k]} = \text{Rec}(L). \quad (4)$$

A reachable word of rank r first occurs at generation

$$d_L(w) = \lceil \log_2 r \rceil. \quad (5)$$

Consequently $T(L) = L$ if and only if every nonempty slice L_n is its full Cartesian product of coordinate projections.

Proof. Iterating Theorem 3.2 gives $\rho_{L^{[k]}}(w) = \lceil \rho_L(w)/2^k \rceil$ for finite ranks. Membership in $L^{[k]}$ is equivalent to rank one relative to that language, which proves the first formula and the depth formula. Finite rank is equivalent to coordinate-hull membership, proving the union formula. The hull is fixed by crossover because each output letter stays in its designated coordinate set. If L is itself fixed, all its iterates equal L and therefore so does their union. The converse follows from the hull's fixedness. $\square$

The proof can also be viewed as a balanced assembly tree. Each leaf supplies one aligned seed fragment, and each internal vertex joins two adjacent fragment blocks. The contraction argument has the advantage of giving a matching lower bound for *every* representation, not merely constructing one efficient derivation.

3.2 There are no missing finite ranks

Theorem 3.4 (No-gap theorem for aligned rank). *If $L_n \neq \emptyset$, the ranks attained on $\text{Rec}(L)_n$ form exactly $\{1, 2, \dots, R_L(n)\}$ for some $R_L(n)$. For $n \geq 1$, $1 \leq R_L(n) \leq n$; for the active zero slice, $R_L(0) = 1$.*

Proof. Let w have rank r , and choose a shortest partition with boundaries $i_0, \dots, i_r$ and corresponding seed witnesses $u_1, \dots, u_r$. For each $1 \leq k \leq r$, form

$$v_k = w[0 : i_{k-1}] u_k[i_{k-1} : n].$$

The first $k - 1$ old fragments followed by the suffix of u_k show that $\rho_L(v_k) \leq k$. Also v_k agrees with w through boundary i_k . If $k \geq 2$ and v_k had a partition into at most $k - 1$ compatible fragments, restrict that partition to its prefix ending at i_k and discard empty intervals. Append the last $r - k$ fragments of the original partition of w . This would give w at most $(k - 1) + (r - k) = r - 1$ fragments, a contradiction. Thus v_k has rank exactly k . The case $k = 1$ follows from positive rank. There are finitely many words of length n , so the largest rank exists. The earlier singleton-fragment bound proves $R_L(n) \leq n$. $\square$

This no-gap assertion concerns aligned seed fragments, not the number of runs in an insertion mask. Its proof changes the output word and retains alignment throughout; it gives no proof of the both-singleton insertion conjecture in the repository.

3.3 Sharp stabilization bounds

Corollary 3.5. *An active slice stabilizes at the exact stage*

$$h_L(n) = \lceil \log_2 R_L(n) \rceil.$$

Empty slices stabilize at stage zero. In particular, for $n \geq 1$ all length- n behavior stabilizes by $\lceil \log_2 n \rceil$. The whole language stabilizes after finitely many generations if and only if its finite ranks are uniformly bounded. If their maximum is $M \geq 1$, the least global stage is $\lceil \log_2 M \rceil$; the empty language has stage zero.

Proof. A slice equals its hull exactly when the rank threshold 2^k covers its largest rank. If it equals its next generation, Lemma 2.2 and Theorem 3.3 imply that it already equals the hull. The global statement follows by taking the supremum of all finite ranks. $\square$

Example 3.6 (Sharpness with a regular seed). Let $L = 0^* \cup 0^*10^*$, the binary words having at most one 1. Its hull is all of $\{0, 1\}^*$, and

$$\rho_L(w) = \max(1, |w|_1), \quad L^{[k]} = \{w : |w|_1 \leq 2^k\}.$$

A compatible fragment contains at most one 1, giving the lower bound. Cutting between consecutive 1's gives the reverse bound, because each resulting fragment extends by zeros to an allowed seed of the same length. Thus $R_L(n) = n$ for $n \geq 1$, and the logarithmic slice bound is sharp. Nevertheless every global inclusion $L^{[k]} \subsetneq L^{[k+1]}$ is strict.

Corollary 3.7 (Shortest possible late word). *Every word in $L^{[k+1]} \setminus L^{[k]}$ has length at least $2^k + 1$. For every $k \geq 0$, Example 3.6 attains this lower bound with 1^{2^k+1} .*

Proof. A newly appearing word has rank greater than 2^k and finite rank at most its length. The example has rank equal to the number of 1's. $\square$

3.4 The frozen-source convention is also exactly solvable

For comparison, let $Y_0 = L$ and $Y_{k+1} = L \otimes_c Y_k$, using both children in Definition 2.1. This convention keeps one parent in the seed at every step. It has the same full closure, but a different clock.

Theorem 3.8 (Frozen-source generation). *For every $k \geq 0$,*

$$Y_k = \{w : \rho_L(w) \leq k + 1\}.$$

Thus a word of finite rank r has frozen-source depth exactly $r - 1$. Finite stabilization is again equivalent to bounded finite rank, and its least stage is $M - 1$ when the maximum finite rank is $M \geq 1$. The empty seed stabilizes at stage zero.

Proof. The assertion at $k = 0$ is the definition of rank one. If the assertion holds at k , restrict a compatible partition of the Y_k parent to the portion inherited by a child. It uses at most $k + 1$ fragments. The inherited portion of the seed parent uses at most one more. Every child therefore has rank at most $k + 2$.

Conversely, suppose w has a compatible partition into $r \leq k + 2$ pieces, with seeds $u_1, \dots, u_r$ and boundaries $i_0, \dots, i_r$. A rank-one word persists by crossing it with itself. For $r \geq 2$, form

$$v = w[0 : i_{r-2}] u_{r-1}[i_{r-2} : n].$$

It has rank at most $r - 1 \leq k + 1$ and agrees with w through boundary i_{r-1} . By induction $v \in Y_k$. Its prefix through that boundary, crossed with the suffix of $u_r \in L$, produces w . This proves the reverse inclusion. The stabilization statements follow because the thresholds $k + 1$ exhaust precisely the finite ranks. $\square$

The distinction is quantitative: the parallel clock is logarithmic in rank, whereas the frozen-source clock is linear. Both clocks detect the same boundedness property. Statements about one convention should not silently be read as finite-stage statements about the other.

4 One-step equality is undecidable for context-free inputs

We use the classical fact that universality of a context-free grammar over a fixed binary alphabet is co-r.e.-complete. Appendix A recalls the usual reduction from Post correspondence and explains the fixed-alphabet step. Here “co-r.e.” means that the negative instances can be effectively enumerated; completeness is under computable many-one reductions.

4.1 A guard construction with universal first generation

Let $\Sigma = \{0, 1\}$, let $@$ be a fresh letter, and put $\Gamma = \{0, 1, @\}$. For any $K \subseteq \Sigma^*$ define

$$G_K = (\Gamma^* \setminus 00\Sigma^*) \cup 00K. \quad (6)$$

This language contains every word involving $@$, every word of length less than two, and every word not beginning with 00 . The only potentially missing words are the words $00x$ with $x \notin K$.

Lemma 4.1 (One-cut repair). *For every $K \subseteq \{0, 1\}^*$,*

$$T(G_K) = \Gamma^*, \quad G_K = \Gamma^* \iff K = \{0, 1\}^*.$$

If K is context-free, a grammar for G_K is effectively constructible. If K is given by an NFA, an NFA for G_K has size polynomial in the input.

Proof. The second equivalence is immediate from the list of missing words. Every word already in G_K remains in $T(G_K)$. A missing word $w = 00x$ has length $n \geq 2$. Define two parents

$$u = w[0 : 1] @ w[2 : n], \quad v = @ w[1 : n].$$

Both contain the guard and hence belong to G_K , both have length n , and crossing after their first letter gives $u[0 : 1]v[1 : n] = w$. Thus every missing word is repaired in one step. The language $\Gamma^* \setminus 00\Sigma^*$ is a fixed regular language; union with the prefix extension $00K$ gives the asserted effective grammar and automaton constructions. $\square$

Theorem 4.2 (Resolution of the one-step decision question). *Given a context-free grammar over the fixed alphabet $\{0, 1, @\}$, deciding whether its language satisfies $L \otimes_c L = L$ is co-r.e.-complete. This remains true on the effective family in which the first crossover generation is the entire free monoid.*

Proof. By Lemma 4.1,

$$T(G_K) = G_K \iff G_K = \Gamma^* \iff K = \{0, 1\}^*.$$

This proves co-r.e.-hardness and the promised restriction. For membership in co-r.e., enumerate pairs of equal-length words and their cut positions. Grammar membership is decidable. Whenever both parents belong to L but their child does not, the enumeration finds a finite witness to failure of closure. Such a witness exists exactly when $T(L) \neq L$, since $L \subseteq T(L)$ always holds. $\square$

The use of 00 is not cosmetic. It ensures that every omitted word has at least two positions, allowing two distinct guard replacements and a genuinely internal repair cut. Without a length guard, a missing empty word or single-letter word cannot be created by aligned crossover from other words of the same length.

Corollary 4.3. *There is no computable function of grammar-description size bounding the length of the shortest counterexample to one-step closure for all nonclosed context-free grammars over this fixed alphabet.*

Proof. Such a bound would allow a finite exhaustive search. For each word length below the bound, enumerate all equal-length parents and cuts and use grammar membership to test the resulting violations. If no violation were found, the bound would certify closure, contradicting Theorem 4.2. $\square$

5 Finite stabilization and an exact block amplification

One-step undecidability does not by itself imply undecidability of eventual stabilization: the guarded examples above *always* stabilize by stage one. We now amplify a single omitted block into arbitrarily high rank.

Introduce a fresh separator $\#$ and put $\Delta = \{0, 1, @, \#\}$. Define

$$B_K = G_K(\#G_K)^*. \quad (7)$$

Thus a word belongs to B_K exactly when every block between successive separators belongs to G_K , including the initial and final blocks. Empty blocks are allowed because $\varepsilon \in G_K$. This construction preserves context-freeness effectively.

Theorem 5.1 (Exact rank of repeated forbidden blocks). *Suppose $x \notin K$, put $w = 00x$, and let $m = |w| \geq 2$. For $r \geq 1$ define*

$$W_r = w(\#w)^{r-1}.$$

Then

$$|W_r| = (m+1)r - 1, \quad \rho_{B_K}(W_r) = r + 1, \quad d_{B_K}(W_r) = \lceil \log_2(r+1) \rceil. \quad (8)$$

Proof. Number the r copies of w . The j th copy occupies the interval between boundaries

$$s_j = (j-1)(m+1), \quad t_j = s_j + m.$$

Consider any compatible-fragment partition of W_r . We claim it must have an internal partition boundary in the integer interval $[s_j, t_j]$ for each j .

If there is no such boundary, one partition fragment contains the entire j th copy of w . When a neighboring separator exists, the fragment also contains that separator, because a cut immediately before or after the copy would be a boundary in $[s_j, t_j]$. At the beginning or end of the whole word, the corresponding end of the seed is fixed by alignment and equal total length. Therefore every complete seed matching this fragment contains w as a whole separator-delimited block. This is impossible in B_K , because $w = 00x \notin G_K$.

The r integer intervals $[s_j, t_j]$ are pairwise disjoint: $s_{j+1} = t_j + 1$. Thus at least r distinct internal partition boundaries are needed, and there are at least $r+1$ fragments.

For the upper bound, place one cut strictly inside each copy of w , for example immediately after its first letter. These cuts give $r+1$ fragments. No resulting fragment contains a whole copy of w . For any one fragment, retain the separator positions of W_r , retain the letters of that fragment, and replace all other nonseparator positions by $@$. In every block at least one position has been replaced, so every block contains a guard and belongs to G_K . The resulting full word lies in B_K and matches the chosen fragment at the required position. Hence the partition is compatible and has $r+1$ pieces. The depth formula follows from Theorem 3.3. $\square$

The lower bound allows seed words with arbitrary separator placements outside the chosen fragment. It does not quietly restrict the minimization to seeds with the same global block layout. The separators *inside* an overlong fragment, together with the actual word endpoints, force the bad block.

Theorem 5.2 (Finite-stabilization undecidability). *It is undecidable, given a context-free grammar over the fixed four-letter alphabet Δ , whether its parallel self-crossover iterates eventually stabilize. This holds even under the promise that the language is either Δ^* or has strict inclusions at every generation. For each fixed $k \geq 0$, deciding $L^{[k]} = L^{[k+1]}$ is co-r.e.-complete.*

Proof. If $K = \{0, 1\}^*$, then $G_K = \Gamma^*$ and $B_K = \Delta^*$, so stabilization occurs at stage zero. Otherwise fix $x \notin K$ and apply Theorem 5.1. For every $k \geq 0$ the word W_{2^k} has rank $2^k + 1$, and therefore

$$W_{2^k} \in B_K^{[k+1]} \setminus B_K^{[k]}, \quad |W_{2^k}| = (m+1)2^k - 1. \quad (9)$$

Thus B_K satisfies the promise and stabilizes if and only if K is universal. This proves undecidability and co-r.e.-hardness for each fixed adjacent equality problem.

For a fixed k , membership in either finite generation is decidable: for a candidate word of length n , one may enumerate the finite set of seed words of length n using grammar membership, then compute crossover $k+1$ times. Enumerating words consequently semidecides a failure of the adjacent equality. Section 6 gives a much faster membership algorithm. $\square$

Corollary 5.3 (The same decision boundary with a frozen source). *Finite stabilization is also undecidable for $Y_{k+1} = L \otimes_c Y_k$, and every fixed adjacent equality problem $Y_k = Y_{k+1}$ is co-r.e.-complete for CFGs. On the nonuniversal side of the block construction, W_{k+1} has frozen-source depth $k+1$ and therefore witnesses strictness at stage k .*

Proof. Theorem 3.8 turns the exact block rank $r+1$ into depth r . Use $r = k+1$ for a fixed-stage witness. The effective reduction and the semidecision of failures are the same as in Theorem 5.2. $\square$

Proposition 5.4 (An arithmetical upper bound). *The set of context-free grammars whose self-crossover sequence eventually stabilizes belongs to Σ_2^0 and is Π_1^0 -hard. No completeness claim at level Σ_2^0 is made here.*

Proof. The condition has the form

$$\exists k \forall w \quad (w \in L^{[k+1]} \implies w \in L^{[k]}),$$

with a decidable matrix, uniformly in the grammar, k , and w . Theorem 5.2 reduces the co-r.e.-complete universality problem to this condition, proving Π_1^0 -hardness. $\square$

6 Polynomial-time rank computation for a fixed target

Global undecidability is compatible with efficient local membership. The aligned-fragment invariant makes this particularly transparent.

Proposition 6.1 (Compatibility by constrained parsing). *Let G be a context-free grammar in Chomsky normal form, of size g , and let w have length $n \geq 1$. All interval-compatibility answers for w can be computed in $O(gn^5)$ time. Consequently $\rho_{L(G)}(w)$ and its first generation, when finite, can be computed in that time. The bound is in the size after the usual polynomial-time grammar normalization.*

Proof. For $0 \leq i < j \leq n$, compatibility is equivalent to

$$L(G) \cap \Sigma^i \{w[i:j]\} \Sigma^{n-j} \neq \emptyset. \quad (10)$$

Run CYK on a length- n string whose positions inside $[i, j)$ are fixed to the corresponding letters of w and whose other positions are wildcards. A terminal production is allowed at a wildcard if it produces any alphabet letter; at a fixed position it must produce that letter. The binary CYK recursion is unchanged. Its acceptance means that some full word meeting these position constraints belongs to the language. Each query costs $O(gn^3)$, and there are $O(n^2)$ intervals.

After the queries, make a directed acyclic graph on boundaries $0, 1, \dots, n$, with an edge $i \rightarrow j$ exactly for the compatible intervals. The least number of edges from 0 to n is exactly the fragment rank. Equivalently,

$$D[0] = 0, \quad D[j] = 1 + \min\{D[i] : 0 \leq i < j, [i, j) \text{ compatible}\},$$

with the minimum of the empty set equal to ∞ . This last step takes $O(n^2)$ time. The depth is obtained from (5). $\square$

This algorithm answers membership in $L^{[k]}$ even when k is given in binary and is very large: compare the computed depth with k , rather than constructing the integer 2^k . A word has finite depth at most $\lceil \log_2 n \rceil$. The empty word is handled by a single grammar membership test. Back pointers in the dynamic program and parsing tables recover a shortest partition and its seed witnesses; balanced pairing then recovers an explicit crossover derivation.

The conservative $O(gn^5)$ bound does not assert optimality. Sharing work between wildcard parses should improve it. More importantly, no step requires a context-free description of an intermediate generation. Such a description has not been assumed to exist.

7 Regular inputs: finite-state tests and limitedness

7.1 One-step closure for deterministic automata

Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a complete DFA with s states. The completion convention includes a rejecting sink when necessary.

Theorem 7.1 (A cubic-state violation graph). *One can decide $T(L(\mathcal{A})) = L(\mathcal{A})$ in $O(s^3|\Sigma|^2)$ time by graph reachability. If closure fails, there is a counterexample word of length at most $2s^3 - 1$.*

Proof. Use states $(p, q, r, e) \in Q^3 \times \{0, 1\}$. The first two components read the two parent words; the third reads the child; e records whether the cut has occurred. Initially all three states are q_0 and $e = 0$. There is a single possible phase change $(p, q, r, 0) \rightarrow (p, q, r, 1)$, consuming no letter.

On a child letter a , choose an arbitrary letter b for the parent whose letter is not inherited. In phase zero update

$$(p, q, r) \mapsto (\delta(p, a), \delta(q, b), \delta(r, a));$$

in phase one update

$$(p, q, r) \mapsto (\delta(p, b), \delta(q, a), \delta(r, a)).$$

A state is accepting for the violation graph when $p, q \in F$ and $r \notin F$. Every accepting path supplies two accepted equal-length parents and a rejected child, with one aligned cut. Conversely every closure violation supplies such a path, using phase changes at the endpoints when needed.

There are $2s^3$ states and $O(s^3|\Sigma|^2)$ transitions. Reachability therefore has the stated complexity. A shortest edge path to an accepting state is simple, has at most $2s^3 - 1$ edges, and consumes no more letters than that. Thus a counterexample of the asserted length exists. $\square$

Ordinary graph search finds a bounded witness; using zero–one breadth-first search, assigning zero to the phase change and one to letter transitions, finds a shortest child word. The graph tracks the two parents independently outside the inherited portions, which is essential for exact-length matching.

Theorem 7.2 (NFA complexity). *For a language given by an NFA, one-step crossover closure is PSPACE-complete, already over a fixed three-letter alphabet.*

Proof. For hardness, apply the polynomial NFA construction in Lemma 4.1 to binary NFA universality. The latter problem is PSPACE-complete; fixed-alphabet hardness is classical, and is also available under stronger automaton restrictions [6].

For membership, first eliminate epsilon transitions. Adapt the violation graph: its parent components choose individual NFA runs, while its child component is the subset of all states reachable on the child prefix. Acceptance requires accepted parent runs and a child subset disjoint from F . A state consists of two NFA states, one subset, and one phase bit, and can be stored in polynomial space. Reachability in the exponentially large implicit graph is decidable in polynomial space. PSPACE is closed under complementation, so the closure rather than violation problem has the same upper bound. $\square$

7.2 Encoding all fragment ranks by a distance automaton

A *distance automaton* is a finite nondeterministic automaton with transition costs 0 or 1. The value of a word is the minimum total cost of an accepting path, or ∞ if there is no accepting path. The classical *limitedness problem* asks whether all finite values are bounded by one integer. It is decidable in PSPACE in the explicit automaton size [5]. We use this theorem, rather than reprove it.

For a complete DFA $\mathcal{A}$ define, for $S \subseteq Q$,

$$\text{Post}(S) = \{\delta(q, a) : q \in S, a \in \Sigma\}, \quad \text{Pre}(S) = \{q : \exists a \in \Sigma, \delta(q, a) \in S\}.$$

For a target length n , set

$$U_i = \delta(q_0, \Sigma^i), \quad V_i = \{q : \delta(q, v) \in F \text{ for some } v \in \Sigma^{n-i}\}.$$

Then $U_0 = \{q_0\}$, $U_{i+1} = \text{Post}(U_i)$, $V_n = F$, and $V_i = \text{Pre}(V_{i+1})$. An interval $w[i : j]$ is compatible exactly when reading it from some state in U_i ends in V_j .

Theorem 7.3 (A finite-state representation of rank). *From an s -state complete DFA for L one can construct a distance automaton with at most $s4^s$ states, allowing a set of initial states, whose finite-word value is*

$$\text{val}(w) = \rho_L(w) - 1$$

when $\rho_L(w) < \infty$, and ∞ otherwise. In particular $\text{Rec}(L)$ is effectively regular.

Proof. A state is a triple (U, V, q) , where $U, V \subseteq Q$ and $q \in Q$. The initial states have $U = \{q_0\}$, $q = q_0$, and arbitrary V . Final states satisfy $V = F$ and $q \in F$.

When consuming the next child letter a , choose a new subset V' subject to $\text{Pre}(V') = V$ and set $U' = \text{Post}(U)$. There are two sorts of transitions:

$$\begin{aligned} (U, V, q) &\longrightarrow (U', V', \delta(q, a)) && \text{at cost 0;} \\ (U, V, q) &\longrightarrow (U', V', \delta(p, a)) && \text{at cost 1, } q \in V, p \in U. \end{aligned}$$

The second transition closes the current fragment and starts a new one just before consuming a .

For any accepting path of length n , the boundary conditions $V_n = F$ and $V_i = \text{Pre}(V_{i+1})$ force the guessed subsets to be exactly the sets V_i defined above. The U_i are forced by their forward recurrence. A completed

fragment starts in a state reachable by an arbitrary prefix of the required length and ends in a state having an arbitrary accepting suffix of the required length. It therefore extends to a genuine seed in L_n . The last fragment ends in F and has the same property.

An accepting path of cost c thus gives a compatible partition into at most $c + 1$ nonempty fragments. A reset before the first letter is redundant: $U_0 = \{q_0\}$, so deleting that reset preserves its state transition and reduces its cost. There are no resets after the last letter, because resets are combined with a consumed letter. A minimum-cost path consequently corresponds to exactly its positive-length fragments.

Conversely, given a compatible partition into r fragments, choose the states and runs of their seed witnesses. Continue at cost zero within a fragment and reset at cost one at each of its $r - 1$ internal boundaries. The accepting path has cost $r - 1$. This proves the equality of values. For ε , a cost-zero accepting initial state exists exactly when $q_0 \in F$, agreeing with the rank convention.

There are $2^s \cdot 2^s \cdot s = s4^s$ states. Forgetting the weights yields an NFA recognizing the finite-rank domain, which is $\text{Rec}(L)$ by Theorem 3.3. $\square$

Corollary 7.4 (Decidable regular-input stabilization). *Finite stabilization of parallel self-crossover, and of frozen-source crossover, is decidable for regular languages. For complete DFA input, the preceding construction and the classical limitedness algorithm give an EXPSPACE upper bound in the DFA size. This is an upper bound, not a claimed optimal complexity.*

Proof. By Corollary 3.5 and Theorem 3.8, either form of stabilization is equivalent to bounded finite rank. Theorem 7.3 converts this into limitedness of a distance automaton. Its explicit state and transition description has size $2^{\text{poly}(s+|\Sigma|)}$; polynomial space in that size is exponential space in the original description size. $\square$

When limitedness holds, the least stabilization stage is also computable. Indeed, each finite iterate of a regular language is effectively regular by a product-and-cut construction, and equality of regular languages is decidable. Generate iterates and test adjacent equality until the first one occurs. The limitedness decision guarantees termination of this second procedure on the bounded side. This gives an effective answer, not a practical implementation of an optimal limitedness solver.

8 Rational counting for the full closure of every CFL

The regularity conclusion of Theorem 7.3 does not extend to context-free inputs. A weaker conclusion does, and it is stronger than rationality of the ordinary length series: all commuting letter weights may be retained.

8.1 Effective semilinear position supports

Lemma 8.1. *Let $L \subseteq \Sigma^*$ be context-free. For each $a \in \Sigma$, the set*

$$E_a = \{(i, j) \in \mathbb{N}^2 : \exists u \in \Sigma^i, v \in \Sigma^j, uav \in L\} \quad (11)$$

is effectively semilinear, and hence effectively definable in Presburger arithmetic.

Proof. Take two tagged copies of the alphabet and a new marker $\bullet$ that is mapped to a . The inverse image of L under the letter-forgetting homomorphism is context-free. Intersect it with the regular language “left-tagged letters, then $\bullet$, then right-tagged letters.” Its Parikh image is effectively semilinear by the effective form of Parikh’s theorem [3]. Sum the counts of left-tagged letters, sum the counts of right-tagged letters, and discard the marker coordinate. The resulting linear image is exactly E_a . $\square$

For $0 \leq i < n$, the letters allowed at position $i + 1$ of a hull word are

$$P_{n,i+1}(L) = \{a \in \Sigma : (i, n - 1 - i) \in E_a\}.$$

Therefore, for each $S \subseteq \Sigma$, the predicate $P_{n,i+1}(L) = S$ is a Boolean combination of effective Presburger predicates. Define the number of positions of each type by

$$A_S(n) = \#\{i : 0 \leq i < n, P_{n,i+1}(L) = S\}. \quad (12)$$

Lemma 8.2 (Eventually affine counts on residue classes). *There are an effectively obtainable period $p \geq 1$ and, for each residue class, a starting length N_r in that class such that simultaneously for all $S \subseteq \Sigma$,*

$$A_S(N_r + pt) = b_{S,r} + d_{S,r}t \quad (t \geq 0), \quad (13)$$

with $b_{S,r}, d_{S,r} \in \mathbb{N}$. Each such progression can also be chosen either entirely active ($L_n \neq \emptyset$) or entirely inactive.

Proof. Each A_S is a Presburger counting function. Woods's counting theorem implies that it is a piecewise quasi-polynomial [4, Theorem 1.10]. In one parameter, beyond a finite threshold the polyhedral partition has only unbounded rays, so the function is an eventual quasi-polynomial. Since $0 \leq A_S(n) \leq n$, each constituent polynomial has degree at most one. Take a common period and a sufficiently large starting length in each residue class for all the finitely many subsets S .

The value at the starting length is an integer $b_{S,r} \geq 0$, and the constant first difference along the progression is an integer $d_{S,r}$. A negative first difference would eventually make the count negative, so $d_{S,r} \geq 0$. These procedures are effective: Presburger quantifier elimination and the counting construction obtain a representation, and all subsequent finite refinements are effective. Appendix B also explains the one-parameter affine conclusion directly.

The length set of a context-free language is effectively semilinear by Parikh's theorem. A semilinear subset of $\mathbb{N}$ is eventually periodic. Refine the same period and thresholds so that activity is constant on each progression. This deals explicitly with empty slices. $\square$

8.2 A rational commutative generating function

For commuting variables $\mathbf{x} = (x_a)_{a \in \Sigma}$ put

$$\text{wt}(w) = \prod_{a \in \Sigma} x_a^{|w|_a}, \quad \Phi_L(z, \mathbf{x}) = \sum_{w \in \text{Rec}(L)} z^{|w|} \text{wt}(w).$$

The series counts distinct words, not derivations or seed representations.

Theorem 8.3 (Rational commutative growth). *For every context-free language L , the series $\Phi_L(z, \mathbf{x})$ is an effectively computable rational function. More precisely, it has a representation*

$$\Phi_L(z, \mathbf{x}) = P(z, \mathbf{x}) + \sum_{r \in \mathcal{R}} \frac{z^{N_r} C_r(\mathbf{x})}{1 - z^p D_r(\mathbf{x})}, \quad (14)$$

where P is a polynomial, $\mathcal{R}$ indexes the eventually active residue classes, and C_r, D_r are products of powers of the nonzero linear forms $\sum_{a \in S} x_a$ for $\emptyset \neq S \subseteq \Sigma$. In particular, along each sufficiently late active arithmetic progression,

$$|\text{Rec}(L)_{N_r+pt}| = C_r(\mathbf{1}) D_r(\mathbf{1})^t, \quad (15)$$

where both constants are positive integers. Inactive progressions have count zero.

Proof. On an active slice the choices of letters at different positions are independent. Hence

$$\begin{aligned} \sum_{w \in \text{Rec}(L)_n} \text{wt}(w) &= \prod_{i=1}^n \left(\sum_{a \in P_{n,i}(L)} x_a \right) \\ &= \prod_{\emptyset \neq S \subseteq \Sigma} \left(\sum_{a \in S} x_a \right)^{A_S(n)}. \end{aligned} \quad (16)$$

Insert (13). For $n = N_r + pt$ this becomes $C_r(\mathbf{x})D_r(\mathbf{x})^t$, with

$$C_r(\mathbf{x}) = \prod_{S \neq \emptyset} \left(\sum_{a \in S} x_a \right)^{b_{S,r}}, \quad D_r(\mathbf{x}) = \prod_{S \neq \emptyset} \left(\sum_{a \in S} x_a \right)^{d_{S,r}}.$$

Summing this geometric series in t gives its summand in (14). The finitely many lengths before the chosen starts, including length zero, supply P . Their coordinate sets and activity are decidable, so this polynomial is effective. Specializing each x_a to one yields (15). $\square$

Corollary 8.4. *The ordinary length generating function of $\text{Rec}(L)$ is effectively rational. Its coefficients satisfy an effectively computable constant-coefficient linear recurrence. If $\text{Rec}(L)$ is infinite, its upper exponential growth rate is*

$$\limsup_{n \rightarrow \infty} |\text{Rec}(L)_n|^{1/n} = \max_{r \in \mathcal{R}} D_r(\mathbf{1})^{1/p}.$$

Proof. Rationality and the recurrence follow by setting $\mathbf{x} = \mathbf{1}$. On an active progression the n th root of $C_r(\mathbf{1})D_r(\mathbf{1})^t$, with $n = N_r + pt$, tends to $D_r(\mathbf{1})^{1/p}$. There are finitely many progressions. Infinitude ensures that at least one is active. $\square$

This is not a rationality theorem for the noncommutative word series and not a regularity theorem for the hull. The generating function has forgotten *where* each coordinate type occurs. Its rational form reflects the eventually affine *number* of positions of each type. The next family shows why that distinction is substantive.

9 A linear context-free seed with a non-context-free closure

9.1 Definition, grammar, and position geometry

Fix an integer $q \geq 3$ and define a binary seed

$$\begin{aligned} S_q &= 0^* \\ &\cup \{0^i 1 0^j : j \geq (q-1)i + q - 1\} \\ &\cup \{0^i 1 0^j : i \geq (q-1)j + q - 1\}. \end{aligned} \quad (17)$$

Every seed has at most one 1. The asymmetric inequalities allow that 1 only in an initial or final portion of the word.

Lemma 9.1. *The language S_q is linear context-free. If $n = qm + r$ with $0 \leq r < q$, then the coordinate hull permits 1 exactly in the first m and last m positions.*

Proof. For the second language in (17), use

$$A \rightarrow 0A0^{q-1} \mid 10^{q-1}Z, \quad Z \rightarrow 0Z \mid \varepsilon.$$

After i recursive uses, the trailing zero count is $(q-1)i + q - 1 + t$, $t \geq 0$, as required. Reversing the productions gives a linear grammar for the third language. Their union with 0^* is linear: each production still has at most one nonterminal.

For a word of length $n = i + 1 + j$, the first inequality is equivalent to $n \geq q(i + 1)$. Thus the allowed position $i + 1$ is at most $\lfloor n/q \rfloor = m$. The second inequality gives the symmetric last m positions. The all-zero seed allows 0 everywhere. $\square$

Write $K_q = \text{Rec}(S_q)$. Lemma 9.1 gives the exact description

$$K_q = \bigcup_{\substack{m \geq 0 \\ 0 \leq r < q}} \{u 0^{(q-2)m+r} v : u, v \in \{0, 1\}^m\}. \quad (18)$$

For example, at length 10 and $q = 3$ the first and last three positions are free, while the middle four positions are zero. There are $2^6 = 64$ hull words, but only seven seed words.

Theorem 9.2 (The closure is not context-free). *For every $q \geq 3$, the language K_q is not context-free, although its seed S_q is linear context-free.*

Proof. Intersect K_q with the regular language $1^+0^+1^+$. A word $1^a0^b1^c$ with $a, b, c \geq 1$ lies in the intersection exactly when

$$(q-1)a \leq b+c, \quad (q-1)c \leq a+b. \quad (19)$$

Indeed, the left block of ones fits in the first $\lfloor n/q \rfloor$ positions exactly when $qa \leq n = a + b + c$, and similarly for the right block.

Suppose this intersection were context-free, with pumping length p . The word

$$z = 1^p 0^{(q-2)p} 1^p$$

satisfies both inequalities at equality. For any admissible pumping factorization, the two pumped portions are contained in one window of length at most p , and their combined length is positive. This window cannot meet both outer blocks of ones, because the intervening zero block has length at least p .

Assume, as the pumping property would require, that pumping both down to zero copies and up to two copies stays in the intersection. If either pumped word fails to have three nonempty blocks in the required order, this already is a contradiction. Otherwise let $\delta_a, \delta_b, \delta_c$ be the numbers of letters removed from the three original blocks by pumping down. The up-pumped word adds these same counts. Apply (19) to both pumped words and use equality for z . Opposite signs force

$$(q-1)\delta_a = \delta_b + \delta_c, \quad (q-1)\delta_c = \delta_a + \delta_b.$$

Subtracting gives $\delta_a = \delta_c$, and then $\delta_b = (q-2)\delta_a$. The window misses at least one outer block, so one of δ_a, δ_c is zero. All three counts are therefore zero, contradicting the positive combined length of the pumped portions. The intersection is not context-free, hence neither is K_q . $\square$

9.2 Every rank, every generation, and exact counts

Theorem 9.3. For $w \in K_q$,

$$\rho_{S_q}(w) = \max(1, |w|_1). \quad (20)$$

Consequently

$$S_q^{[k]} = K_q \cap \{w : |w|_1 \leq 2^k\}. \quad (21)$$

For $n = qm + r$, $0 \leq r < q$,

$$|(K_q)_n| = 4^m, \quad (22)$$

$$|(S_q^{[k]})_n| = \sum_{j=0}^{2^k} \binom{2m}{j}, \quad (23)$$

where $\binom{2m}{j} = 0$ for $j > 2m$. The exact slice stabilization stage is $\lceil \log_2 \max(1, 2m) \rceil$.

Proof. Any fragment matching a seed contains at most one 1, so rank is at least $\max(1, |w|_1)$. Partition w between its consecutive ones. Each fragment then has at most one 1; if it has one, that position is seed-supported by Lemma 9.1. Fill all other positions of the full word with zero to extend the fragment to a seed. A fragment with no ones extends to the all-zero seed. This proves (20), including the all-zero and empty cases under the stated convention.

There are $2m$ free binary coordinates. Choosing which j of them are one gives $\binom{2m}{j}$ words. Sum over all j for the hull and over $j \leq 2^k$ for generation k . The maximum rank is $\max(1, 2m)$, giving the last assertion. $\square$

Marking each 1 by a variable x yields

$$\sum_{w \in K_q} z^{|w|} x^{|w|_1} = \frac{1 + z + \cdots + z^{q-1}}{1 - (1+x)^2 z^q}. \quad (24)$$

In particular,

$$\sum_{n \geq 0} |(K_q)_n| z^n = \frac{1 + z + \cdots + z^{q-1}}{1 - 4z^q}. \quad (25)$$

The rational series is exceptionally simple, even though the underlying language is not context-free.

9.3 Sharp recurrence orders of the finite generations

Theorem 9.4 (Exponential growth of minimal recurrence order). Fix $q \geq 3$ and $k \geq 0$, and write $D = 2^k$. The ordinary length generating function of $S_q^{[k]}$, in reduced form over $\mathbb{Q}$, has denominator

$$(1 - z)(1 - z^q)^D. \quad (26)$$

Its minimal eventual constant-coefficient recurrence has order $qD + 1 = q2^k + 1$. For each fixed k , its coefficients grow polynomially of degree D , while the full hull has exponential growth rate $4^{1/q}$.

Proof. The polynomial

$$b_D(m) = \sum_{j=0}^D \binom{2m}{j}$$

has degree exactly D and leading coefficient $2^D/D!$. Interpreting each binomial as a polynomial in m agrees with the combinatorial convention at all $m \in \mathbb{N}$, since the falling factorial vanishes when $j > 2m$. Consequently

$$\sum_{m \geq 0} b_D(m) t^m = \frac{Q_D(t)}{(1-t)^{D+1}}, \quad \deg Q_D \leq D, \quad Q_D(1) = 2^D \neq 0.$$

One proof is to expand $b_D(m)$ in the binomial-polynomial basis; the coefficient of $\binom{m}{D}$ is $D! \cdot 2^D/D! = 2^D$, and only that term contributes to the numerator at $t = 1$.

Lengths $qm, \dots, qm + q - 1$ have the same coefficient. Thus

$$\begin{aligned} \sum_{n \geq 0} |(S_q^{[k]})_n| z^n &= (1 + z + \dots + z^{q-1}) \frac{Q_D(z^q)}{(1 - z^q)^{D+1}} \\ &= \frac{Q_D(z^q)}{(1 - z)(1 - z^q)^D}. \end{aligned}$$

All roots of the displayed denominator satisfy $z^q = 1$, where its numerator is $Q_D(1) \neq 0$. Hence it is reduced and its degree $qD + 1$ is the minimal recurrence order. Finally $b_D(\lfloor n/q \rfloor) \sim (2^D/(D!q^D))n^D$, whereas $|(K_q)_n| = 4^{\lfloor n/q \rfloor}$ has exponential rate $4^{1/q}$. $\square$

For the frozen-source sequence of Theorem 3.8, the same calculation holds with $D = k + 1$ rather than $D = 2^k$. Its generation- k minimal recurrence order is therefore $q(k + 1) + 1$.

The finite-generation and full-closure limits therefore do not commute at the level of exponential growth. Every fixed generation has growth rate one, but their increasing union has larger growth rate. This supplies a concrete warning against extrapolating an infinite closure from a few rational finite-stage enumerators.

q	generation k	rank threshold 2^k	polynomial growth degree	recurrence order
3	0	1	1	4
3	1	2	2	7
3	2	4	4	13
3	3	8	8	25
4	2	4	4	17
5	3	8	8	41

10 Verification, reproducibility, and proof boundaries

10.1 Executed exact audits

The accompanying standard-library Python program performs independent finite checks using explicit source sets, direct crossover, interval dynamic programming, finite-state paths, and integer counting. The delivered run completed **1,766,639 checks**, all passing. The JSON receipt records the detailed counts; these are finite audits, not universal proof certificates.

Audit	Executed scope
All finite binary slices	All 65,814 seed subsets at lengths 0, 1, 2, 3, 4; 1,050,698 target-rank queries. Checks exact contraction, coordinate hull, every finite rank, generation thresholds, frozen-source generations, and stabilization.
All two-state binary DFAs	All 64 complete transition/final-state choices with fixed initial state; 8,128 target queries at lengths 0 through 6. Compares source enumeration, DFA fragment rank, and the distance-automaton cost.
Guard repair	All 255 omitted binary patterns 00x of lengths 2 through 9; checks the actual same-length parent repair.
Block amplification	Ranks and distances of repeated forbidden block 00 for 1 through 64 blocks. Independent enumeration over the full four-letter alphabet at lengths 2, 5, 8.
The linear-seed family	$q = 3, \dots, 7$, all binary target words through length 12; exact ranks, weighted histograms, and finite-stage counts.
Recurrence checks	25 parameter pairs, $q = 3, \dots, 7$ and $k = 0, \dots, 4$; integer convolution with the proved denominator and nonzero top finite differences.

For the distance-automaton audit, the accepting boundary condition forces a unique sequence of right subsets V_i . The program computes that sequence backward rather than enumerate unsuccessful guesses. It then evaluates the cost transitions separately from interval-partition dynamic programming. This checks the accepted-path cost semantics; it does not implement or test the general limitedness algorithm imported from the literature.

The block test uses the regular specialization $G = \{0, 1, @\}^* \setminus \{00\}$. This does not prove the grammar undecidability reduction. It checks the combinatorial amplification while allowing arbitrary source separator placements, including the placements that could invalidate a careless lower-bound proof.

10.2 Files and rebuilding

The main source is standalone and has its bibliography internally. The package layout is

```
ProveIt_Constrained_Crossover/
  article.tex
  article.pdf
  README.md
  Makefile
  code/verify.py
  data/verification.json
  data/verification.txt
  notes/provenance.md
  notes/proof-audit.md
```

From the package directory, run

```
python3 code/verify.py
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

The verifier uses no external packages and no floating-point mathematics; the only floating-point output is elapsed wall-clock time. Assertions are implemented as explicit exception-raising checks and remain active under `python -O`. The LaTeX build uses commonly distributed packages, including `newtxtext` and `newtxmath`.

[write] *Shipped layout*. In the collection the files live at the report root,

```
SetTheory/Cardinals/docs/reports/
automata-and-formal-languages/constrained-crossover-closure/.
```

There, `notes/provenance.md` and `notes/proof-audit.md` are shipped as `provenance.md` and `proof-audit.md`, and `Makefile` as `code/Makefile`; the other paths are unchanged, and all these files are byte-identical to the delivery. The delivered README and 23-page PDF are not shipped: the report's README replaces the former, and `article.pdf` is a build of this text. The verifier writes `../data/verification.json` relative to its own directory, so run in place it overwrites the shipped receipt; the README gives commands for a copy. The build now also uses the `shuffle` package, for the sibling report's operation in Section 1.4.

10.3 What was not verified or claimed

No Lean or Rocq kernel has checked the proofs. The external Parikh, Presburger, and distance-limitedness theorems are explicitly cited mathematical inputs, not imported formal declarations. No general CFG-to-Presburger compiler, limitedness solver, or full symbolic rational-enumerator implementation is included. The polynomial-time grammar rank algorithm is proved but the executable reference verifier specializes to finite seeds and DFAs.

The novelty search inspected the cited Hughes version, the relevant ProveIt source and collection catalogue, and targeted literature on crossover, universality, effective Parikh images, Presburger counting, and distance automata. It did not identify a prior resolution of the specific constrained one-step question or an equivalent package of the quantitative results. Search coverage is not an exhaustive bibliography and is not a proof of priority. In particular, the product-hull observation and possible equivalent rank formulations deserve review by specialists in recombination systems.

[write] Placement in ProveIt, a repository with Lean and Rocq developments, confers no formal status on this report, and none of its statements is formalized. ProveIt has no formal development of crossover, aligned rank, context-free grammars, Post correspondence, Parikh images, Presburger counting or distance-automaton limitedness. The nearest formal neighbour is `Logic/PresburgerArithmetic`, whose Lean development proves Cooper quantifier elimination for Presburger arithmetic over $\mathbb{Z}$ (`PresburgerArithmetic.Formula.holds_iff_quantifierEliminate` and the decision procedure `PresburgerArithmetic.Formula.presburgerArithmetic_decidable`, in `Lean/PresburgerArithmetic/Decision.lean`), with an independent Rocq proof of the one-variable Cooper step. The effectivity argument of Lemma 8.2 and Appendix B invokes quantifier elimination of this kind but is not connected to that development, and the counting theorem it needs is not formalized anywhere in ProveIt. The Lean plan of Section 11.1 is a plan, not an executed development.

11 Further research questions and a formalization plan

The following are proposed research tasks relative to the present results. Only the Hughes question identified in Section 1 is assigned a specific published open-problem provenance. The rest are not asserted to be newly discovered or globally open without further literature review.

Question 11.1 (Exact arithmetical complexity). Is finite stabilization of a context-free seed Σ_2^0 -complete? The present proof gives a Σ_2^0 upper bound and Π_1^0 -hardness, not a matching lower bound. A stronger construction must encode a genuinely existential bound rather than just universality of one grammar.

Question 11.2 (Sharp regular-input complexity). What is the optimal complexity of deciding finite stabilization for DFA and NFA inputs? Can the $s4^s$ distance-automaton construction be represented more economically,

or can its special two-sided subset structure yield a better algorithm than generic limitedness? Matching hardness results would determine whether the EXPSPACE upper bound is loose.

Question 11.3 (Two-letter undecidability). Can Theorems 4.2 and 5.2 be proved over a binary alphabet? Fixed-length coding does not automatically preserve one-step crossover: a cut inside a codeword can create spurious children. A successful binary encoding must control such internal cuts, not merely preserve language universality.

Question 11.4 (Grammar classes preserving context-freeness). Which context-free seeds have a context-free coordinate hull? The linear counterexample rules out linearity as a sufficient condition. Restrictions on the position-support sets E_a , rather than just grammar ambiguity or linearity, may provide a useful criterion. Deterministic and visibly pushdown subclasses are natural places to investigate.

Question 11.5 (Faster target-word algorithms). Can the $O(gn^5)$ wildcard-parsing method be reduced to one shared chart computation of cubic or near-cubic complexity? A useful implementation should return a shortest fragment partition and full seed witnesses, not just a membership bit, so that crossover derivations can be independently checked.

Question 11.6 (Which rational counting series occur?). Characterize the commutative series realized by context-free recombination closures. Theorem 8.3 forces eventual products of linear forms along arithmetic progressions. This is more restrictive than arbitrary rational nonnegative series. Which compatibility constraints on the semilinear position supports are necessary and sufficient for realization?

Question 11.7 (Restricted cuts and multipoint crossover). For cut positions restricted by a regular or Presburger condition, what replaces aligned fragment rank? For crossover with several cuts between the same two parents, different fragments are coupled by parent reuse; simply replacing 2^k by a larger exponential is not justified without a new proof.

Question 11.8 (Quantitative transition windows). For S_q , the proportion reached by generation k at length $qm + r$ is $4^{-m} \sum_{j \leq 2^k} \binom{2m}{j}$. Study uniform asymptotics when k grows with m , rather than keeping k fixed. The explicit binomial law gives a tractable test case for crossover depth distributions in less rigid seeds.

Question 11.9 (Sharp automaton witness bounds). Determine the best possible shortest-violation bound for an s -state DFA. The $2s^3 - 1$ bound comes from a direct triple-state graph. It may not be sharp because the three components share one transition structure. Explicit lower bound families and state-complexity analyses would be informative.

[write] *Bearing on Hughes's Sections 13–14*. Hughes asks, for his insertion and shuffle iterations, for the exact arithmetical complexity of finite convergence, for restricted language families on which it becomes decidable, for whether the least stabilization index can be computed, and for a general “rank condition” behind his separator proofs; his Section 13 asks for which monotone operators eventual reaching of a fixed point is decidable. For constrained self-crossover, which is monotone and inflationary (Lemma 2.2), the results above give the following counterparts. Single-step and each fixed adjacent equality are co-r.e.-complete for grammars (Theorems 4.2 and 5.2); eventual stabilization is in Σ_2^0 and Π_1^0 -hard (Proposition 5.4), and Question 11.1 asks for the exact level. Regular inputs make stabilization decidable, and the least stage is then computable (Corollary 7.4 and the paragraph after it); every slice stabilizes by stage $\lceil \log_2 n \rceil$ (Corollary 3.5). Aligned-fragment rank with the separator amplification of Theorem 5.1 is a rank-based separator argument for this one operation, not the general condition Hughes asks for. None of this answers his questions for insertion or shuffle.

11.1 A realistic Lean development

A formal development can begin with fixed-length words as functions $\text{Fin}(n) \rightarrow \Sigma$, avoiding casts between lists of different lengths. A cut is a boundary $i \leq n$, and its child chooses the first parent before i and the second afterwards. A compatible partition is a strictly increasing finite boundary list, with one seed witness per interval.

The first milestones should be, in order, the two fragment-transfer lemmas used in Theorem 3.2, the exact iterate formula, the no-gap prefix-replacement argument, and the block-boundary charging lemma. These are finite combinatorial statements independent of grammars and automata, and would establish the core trusted interface.

A second layer can define an executable compatibility matrix and prove that the boundary shortest-path program computes the least rank. The finite reference tests can then be replaced by proved checkers over this interface. For the decision-theoretic reductions, a separate grammar layer must certify regular union, prefixing, concatenation, and star constructions and connect them to a formally stated universality theorem. The present paper does not claim that such a bridge already exists in ProveIt.

The counting theorem requires substantially different infrastructure: effective semilinearity of the marked grammar, Boolean Presburger operations, and the affine residue-class counting lemma. Its dependencies should be tracked separately rather than presenting the whole article as formalized because its finite combinatorics has been checked.

12 Conclusion

Constrained crossover is length-preserving, but length preservation does not make its global grammar problems decidable. A one-letter guard repairs any suitably padded omission in one generation, settling the specific one-step question. Repeated separator-delimited omissions then give rank exactly one more than the number of bad blocks, separating local logarithmic convergence from global finite stabilization.

The aligned-fragment invariant unifies the structural and algorithmic results. It computes exact depth, forces gap-free rank spectra, provides a polynomial local membership procedure, and turns regular-input stabilization into distance-automaton limitedness. At the counting level, semilinear coordinate supports imply rational commutative growth for every context-free seed, but not context-freeness of the generated language. The explicit linear family demonstrates this separation and gives exact recurrence orders at every finite generation.

The principal proofs stand independently of the finite experiments. The experiments supply a reproducible adversarial audit of their finite mechanisms. Independent proof review, priority checking, and formalization are the next steps before treating the draft as a settled research publication.

A The classical universality input

For completeness, here is the standard route to the co-r.e.-completeness used in Section 4. The foundational undecidable problem is Post correspondence [7]. Take two morphisms $h, g : I^* \rightarrow A^*$ and ask whether some $x \in I^+$ satisfies $h(x) = g(x)$. The set of positive instances is recursively enumerable, and the usual effective computation-history reduction gives r.e.-completeness.

Use disjoint alphabets I, A and a separator $\$$. Among encodings $x\$y^R$, where $x \in I^+$ and $y \in A^*$, the

language

$$\{x\$y^R : y \neq h(x)\}$$

is context-free. A pushdown automaton pushes the successive output blocks of h while reading x , then compares the reversed candidate output against the stack. It accepts a symbol mismatch, a short candidate, or a long candidate; the regular encoding conditions can be imposed separately. The analogous language for g is also context-free. Their union, together with all malformed encodings, is universal exactly when the correspondence instance has no solution. This proves co-r.e.-hardness of grammar universality. Its complement is r.e. because grammar membership is decidable and nonmembers can be searched for by length.

To obtain a binary alphabet, map each letter of the finite encoding alphabet to a distinct binary block of one common length ℓ . For a language H over that alphabet, set

$$H' = c(H) \cup (\{0, 1\}^* \setminus c(A_0^*)),$$

where A_0 denotes the entire encoding alphabet. The image $c(H)$ is context-free and $c(A_0^*)$ is regular. The injective fixed-length code gives $H' = \{0, 1\}^*$ exactly when $H = A_0^*$. This is a reduction for universality; as Section 11 emphasizes, it is *not* automatically a reduction preserving crossover.

B Why the positional counts are eventually affine

The special case of Presburger counting needed here also has a direct one-dimensional explanation. After quantifier elimination, a predicate in variables (n, i) is a Boolean combination of linear inequalities and fixed modular congruences. We also impose $0 \leq i < n$.

Refine the residue class of n modulo all congruence moduli, and refine the residue class of i similarly. On these classes, congruences have constant truth values. Each nontrivial inequality in i changes truth value at a rational-affine threshold in n . The order of finitely many such thresholds is fixed once n is sufficiently large: two affine functions either agree identically or their order changes at most once.

Thus the allowed positions eventually form a fixed finite union of intervals between affine thresholds, possibly with selected endpoint positions, in each fixed residue class of i . Counting that class inside such an interval is a difference of floor or ceiling functions of rational-affine expressions. Refine the period of n to absorb the finitely many denominators. Each such count then is affine on a progression. Finite union and disjoint refinement preserve that property.

This argument explains both effectivity and the degree-one conclusion of Lemma 8.2; the general counting theorem of Woods supplies a systematic framework. Since counts are nonnegative for all sufficiently large parameters and integer-valued, their progression increments are nonnegative integers. That final observation is what makes the geometric factors in Theorem 8.3 polynomials rather than Laurent expressions.

C A compact proof-dependency map

Result	Essential inputs
Exact contraction and generation	Definition of aligned crossover; restriction and pairing of consecutive intervals. No grammar theorem.
One-step CFG undecidability	One-cut guard repair; classical fixed-binary CFG universality.
Eventual CFG undecidability	Exact generation; disjoint boundary intervals for forbidden blocks; effective CFL union, concatenation, and star.
Regular-input stabilization	Exact generation; explicitly constructed distance automaton; classical limitedness decidability.
Rational weighted hull series	Effective Parikh semilinearity; one-parameter Presburger counting; coordinate-product enumeration.
Non-CFL linear-seed example	Explicit grammar; elementary position inequalities; CFL pumping lemma and intersection with a regular language.
Minimal finite-stage recurrence	Exact binomial count; degree of a polynomial; reduced rational denominators.

D [write] Provenance of this report

One manuscript. This report is built from a single manuscript, manuscript 02 of batch 44 of ProveIt’s incoming-report intake: *Aligned Fragments and Constrained Crossover*, dated 29 September 2026, delivered as the archive `ProveIt_Constrained_Crossover.zip`, which arrived in commit `ae28ea2db` and was placed in commit `203016015`. Its pin is ProveIt commit `9b24a3a8d545af9624f6ac455f5b548be62818b6` (Section 1). The manuscript contributed everything outside the text marked [write]: every result, proof, example, remark, question, limitation and the Lean plan, all printed. The package’s audit notes and verifier are shipped unchanged beside this article (Section 10.2 lists the paths). Its 23-page US-letter PDF and its README are not shipped; they survive in the arrival commit.

What the write added. Everything marked [write]: the title-page note; in Section 1, where Hughes poses the question, the state of the sibling source, and Sections 1.4 and 1.5; the note after Remark 2.3; the notes on the shipped layout and on formal status in Section 10; the note on Hughes’s Sections 13–14 after the research questions; and this appendix. The locations in Hughes’s preprint were checked against its arXiv v1 PDF when the report was written.

Choices. There was no merge, so no result had to be chosen over another. Every label received the prefix `ccc:`, and every reference was updated with it. Labels were added only on new text and on Question 11.1 and Section 11.1; no theorem, section or equation number of the manuscript changed. The page size stays US letter, as delivered. No symbol was renamed; the `shuffle` package was added for the symbol $\sqcup$ of the sibling report.

References

- [1] Charles E. Hughes. *Undecidability of Adjacent Equality for Insertion, Shuffle, and Crossover Language Operations*. arXiv:2608.27755v1, 27 August 2026. Definition on printed p. 2; the constrained one-step question on printed p. 3. [write] Section 2 (definition), Section 4 (the question, pp. 3–4), Section 14 (open directions, p. 12). <https://arxiv.org/abs/2608.27755v1>.

- [2] Vladimir Reshetnikov and ProveIt contributors. *ProveIt: Gaps in Bounded-Shuffle Hierarchies*, Parts I–II. Inspected 29 September 2026 at the revision and path specified in Section 1. AI-assisted research drafts. [write] The sibling report `insertion-degree-spectra` of this collection (Section 1.4).
<https://github.com/VladimirReshetnikov/ProveIt>.
- [3] Javier Esparza, Pierre Ganty, Stefan Kiefer, and Michael Luttenberger. Parikh’s theorem: A simple and direct automaton construction. *Information Processing Letters* **111**(12) (2011), 614–619.
<https://arxiv.org/abs/1006.3825>. doi:10.1016/j.ipl.2011.03.019.
- [4] Kevin Woods. Presburger arithmetic, rational generating functions, and quasi-polynomials. *Journal of Symbolic Logic* **80** (2015), 433–449. <https://arxiv.org/abs/1211.0020>. doi:10.1017/jsl.2015.4.
- [5] Daniel Kirsten. Distance desert automata and the star height problem. *RAIRO—Theoretical Informatics and Applications* **39**(3) (2005), 455–509. https://www.numdam.org/item/ITA_2005__39_3_455_0/. doi:10.1051/ita:2005027.
- [6] Markus Krötzsch, Tomáš Masopust, and Michaël Thomazo. Complexity of universality and related problems for partially ordered NFAs. *Information and Computation* **255**, Part 1 (2017), 177–192. Preprint arXiv:1609.03460v2. <https://arxiv.org/abs/1609.03460>.
- [7] Emil L. Post. A variant of a recursively unsolvable problem. *Bulletin of the American Mathematical Society* **52** (1946), 264–268.